

IT-Security

Chapter 2: Encryption



1. Symmetric encryption
2. Authenticated encryption
3. Base64 encoding
4. Asymmetric encryption
5. Practical aspects of encryption
6. Key management
7. Random numbers
8. Quantum cryptography



What do we want to learn?



- ▶ What are the types of encryption?
- ▶ What are the most important algorithms and procedures?
- ▶ How can I use encryption in practice?
- ▶ What do I have to pay attention to so that encryption really brings more security?



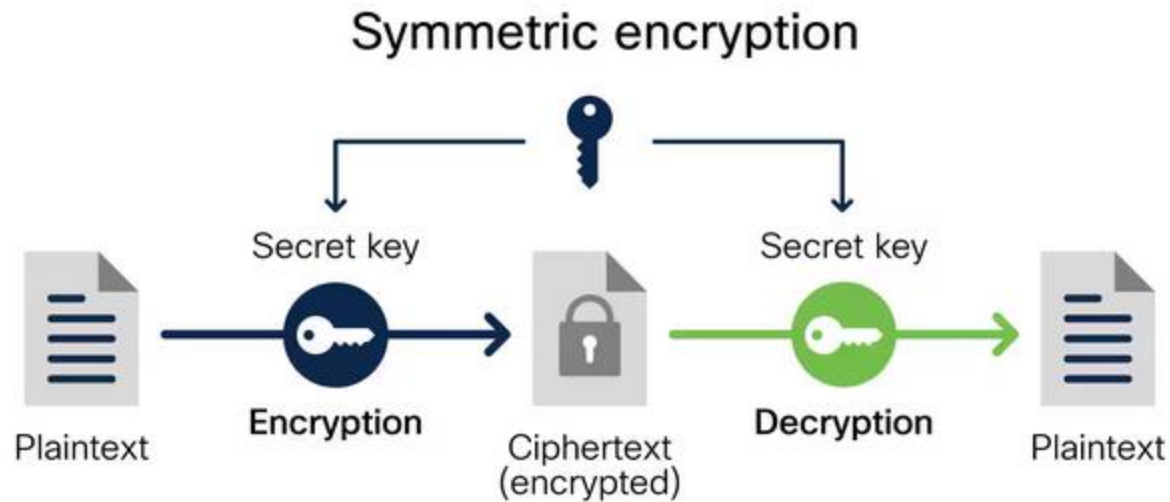
Chapter 2.1: Symmetric Encryption

- Stream Ciphers
- Block Ciphers
- AES
- ECB, CBC
- CTR, PBE



Symmetric encryption

- ▶ One central key for encryption and decryption
- ▶ Disadvantages: exchange of secret key required
- ▶ Advantages:
 - ▶ Very fast,
 - ▶ implementable in HW



Quelle: <https://www.cisco.com/c/en/us/products/security/encryption-explained.html#~q-a>

▶ Symmetric Cipher Variants

▶ Stream ciphers

- ▶ Take a key of fixed length
- ▶ Generate a data stream of arbitrary length, consisting of pseudo-random numbers, from the key
- ▶ Convert each bit of the plaintext to ciphertext using XOR bit by bit
- ▶ Examples: RC4 (UNSECURE since 2013!!!), ChaCha20

▶ Block ciphers

bit blocks

- ▶ Process plaintext and ciphertext in blocks (e.g. 64 Bit, 128 Bit)
- ▶ Have a mode (block cipher mode)
- ▶ Examples: AES, Twofish, Serpent

▶ Padding

- ▶ Filling up the last incomplete block with "regular" patterns.
- ▶ Reason: Some algorithms (mostly block ciphers) require full blocks
- ▶ Examples: PKCS5 padding, W3C padding, ISO padding, ESP padding.

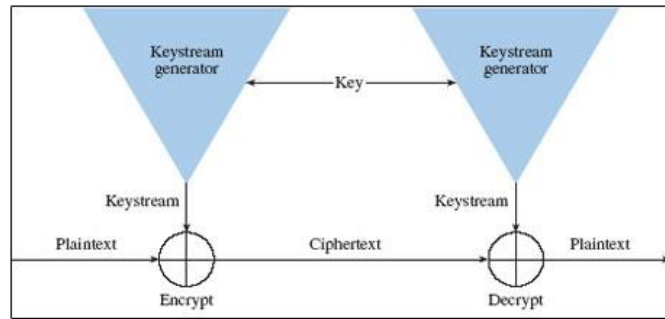
irregular but
reconstructable



Stream cipher variants

► Synchronous stream ciphering

- Generates the **key stream independently** of the clear text or key text

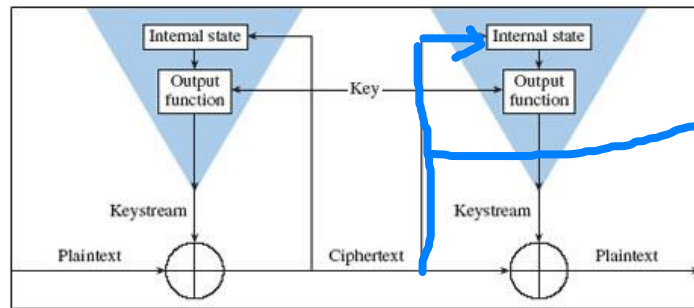


faster because independent/ parallel

From Schneier, 1996, Figure 9.6

► Self-synchronizing stream ciphering

- Key stream **depends on previous encrypted bit**



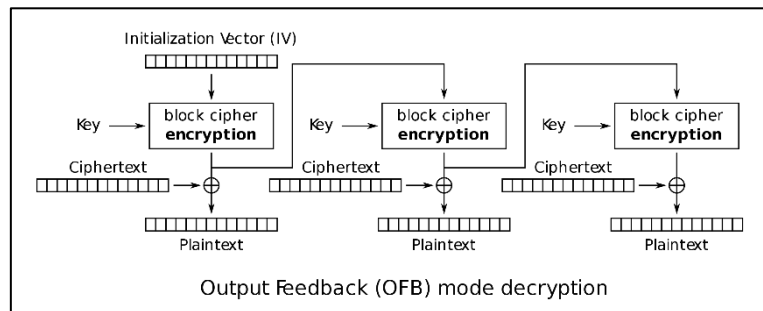
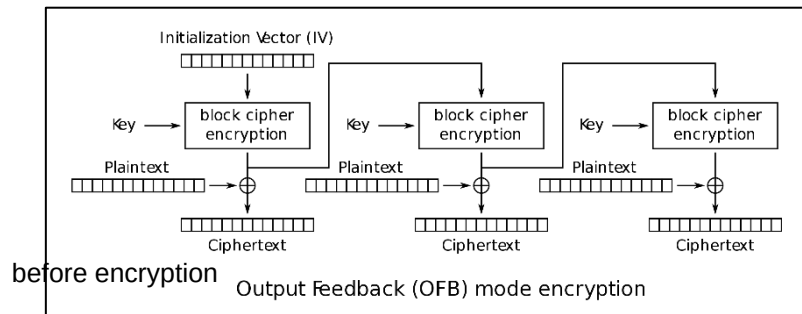
feedback loop
more secure but sequential

From Schneier, 1996, Figure 9.8



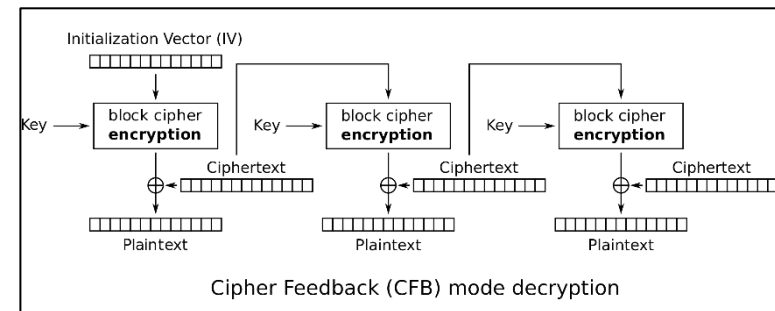
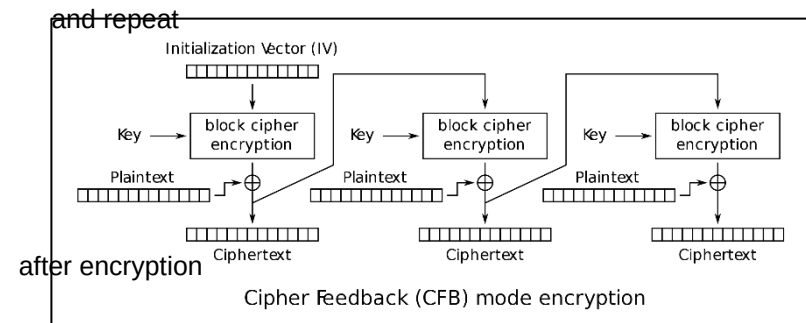
Block ciphers can be implemented as stream ciphers

- ▶ **OFB, Output Feedback Mode**
- ▶ Is a synchronous stream ciphering mode
- ▶ The key stream can be precalculated
encrypted out is returned as feedback instead of actual cipher meaning key stream can be precalculated



https://de.wikipedia.org/wiki/Output_Feedback_Mode

- ▶ **CFB Cipher Feedback Mode**
- ▶ Self-synchronizing stream ciphering
- ▶ Errors in IV or ciphertext affect only two blocks
IV encrypted, output bits divided into sets, left set combined with plaintext, XOR operation applied, result is given to shift register, then encrypted again



https://de.wikipedia.org/wiki/Cipher_Feedback_Mode



The standard encryption method

Advanced Encryption Standard (AES)

▶ Requirements for AES in competition from NIST 2001

- ▶ Block cipher with 128, 192 and 256 bits
- ▶ Mathematical justification of security
- ▶ Simplicity of design
- ▶ Flexibility in block sizes and key lengths
- ▶ Efficiency
- ▶ Easy to implement in HW and SW

Old standard, unsafe today, do not use anymore!!!

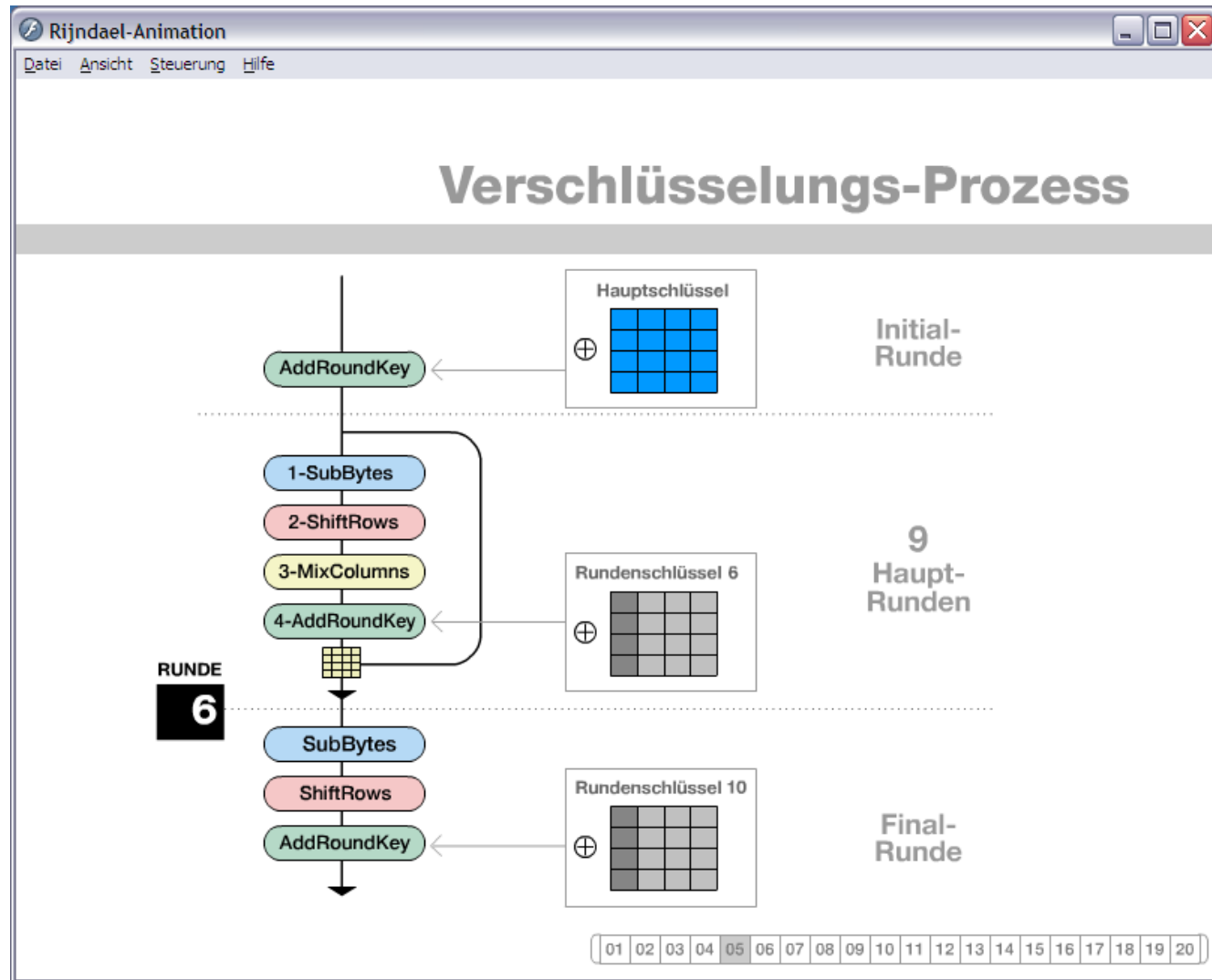
▶ Official successor to DES

- ▶ AES is based on Rijndael algorithm
- ▶ Selected from 15 proposals in an open process
- ▶ All design criteria published
- ▶ Encrypts blocks with fixed length of 128 bits (16 bytes)
- ▶ Key length optionally 128, 192 or 256 bits

recommended key length



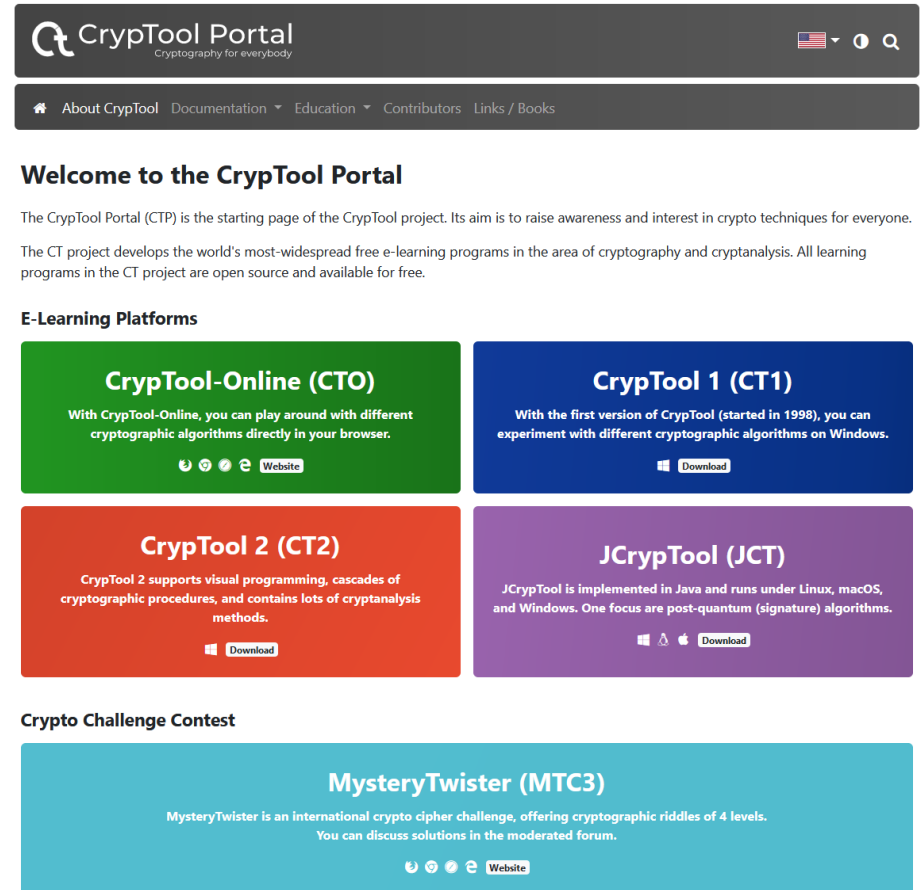
The Algorithm of AES



Quelle: Cryptool Portal,
<https://www.cryptool.org/en/cto/aes-animation>

Small exercise: View simulation of AES

- ▶ Go to CryptTool Portal <https://www.cryptool.org/en/>
- ▶ The CT project develops free e-learning programs in the area of cryptography and cryptanalysis
- ▶ Prof. Bernhard Esslinger, University of Siegen
- ▶ Go to CryptTool-Online (CTO)
Go to AES Animation
<https://www.cryptool.org/en/cto/aes-animation>
- ▶ Go through the animation step by step



The screenshot shows the CryptTool Portal website. The header includes the logo "CryptTool Portal" with the tagline "Cryptography for everybody", a language selector (US flag), and a search icon. Below the header is a navigation bar with links: "About CryptTool", "Documentation", "Education", "Contributors", and "Links / Books". The main content area starts with a "Welcome to the CryptTool Portal" section, followed by a description of the project's aim. Below this is the "E-Learning Platforms" section, which contains four colored boxes: "CryptTool-Online (CTO)" (green), "CryptTool 1 (CT1)" (blue), "CryptTool 2 (CT2)" (orange), and "JCrypTool (JCT)" (purple). Each box provides a brief description and a "Download" or "Website" link. At the bottom is the "Crypto Challenge Contest" section, featuring "MysteryTwister (MTC3)" with a description and a "Website" link.

CryptTool Portal
Cryptography for everybody

About CryptTool Documentation Education Contributors Links / Books

Welcome to the CryptTool Portal

The CryptTool Portal (CTP) is the starting page of the CryptTool project. Its aim is to raise awareness and interest in crypto techniques for everyone.

The CT project develops the world's most-widespread free e-learning programs in the area of cryptography and cryptanalysis. All learning programs in the CT project are open source and available for free.

E-Learning Platforms

CryptTool-Online (CTO)
With CryptTool-Online, you can play around with different cryptographic algorithms directly in your browser.
[Website](#)

CryptTool 1 (CT1)
With the first version of CryptTool (started in 1998), you can experiment with different cryptographic algorithms on Windows.
[Download](#)

CryptTool 2 (CT2)
CryptTool 2 supports visual programming, cascades of cryptographic procedures, and contains lots of cryptanalysis methods.
[Download](#)

JCrypTool (JCT)
JCrypTool is implemented in Java and runs under Linux, macOS, and Windows. One focus are post-quantum (signature) algorithms.
[Download](#)

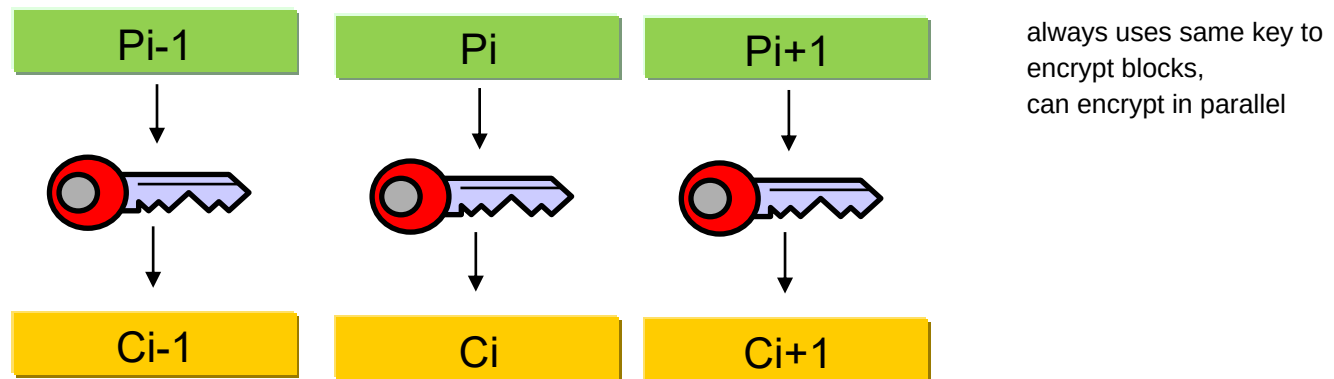
Crypto Challenge Contest

MysteryTwister (MTC3)
MysteryTwister is an international crypto cipher challenge, offering cryptographic riddles of 4 levels. You can discuss solutions in the moderated forum.
[Website](#)



Operating mode for block cipher: ECB

- ▶ A symmetric method (e.g., AES) can be operated in different modes
- ▶ **Electronic Code Book (ECB)**
 - ▶ A plaintext block is encoded into a ciphertext block
 - ▶ Same plaintext blocks generate same ciphertext blocks
 - ▶ All blocks can be ciphered independently of each other parallel
 - ▶ Vulnerable for cryptanalysis (stereotyped beginnings and endings)
 - ▶ Simple bit errors have no influence on other blocks
 - ▶ If bits are lost, resynchronization of block boundaries is required

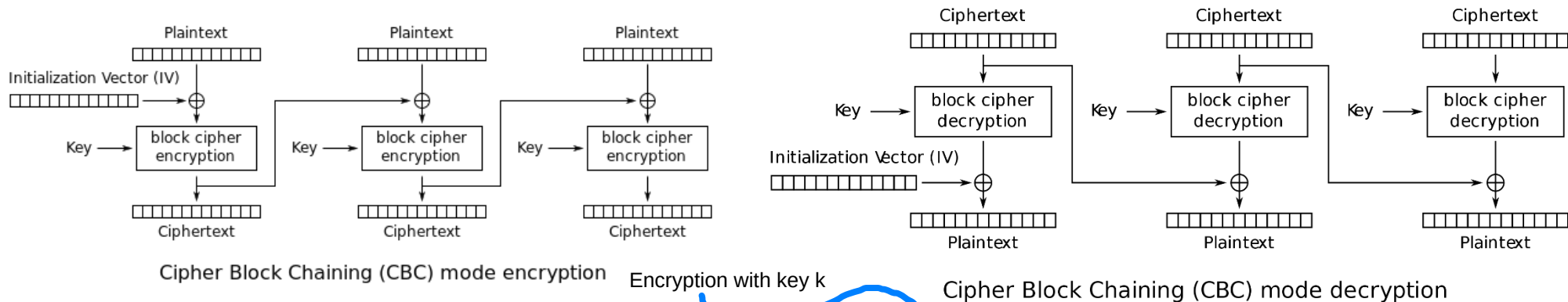




Operating mode for block cipher: CBC

Cipher Block Chaining (CBC)

- ▶ **Feedback:** encryption of a block depends on predecessor blocks
- ▶ Plaintext is XORed with previous ciphertext block before encryption
- ▶ **Initialization vector** for first block necessary:
not secret, uniquely reconstructable for decryption, different for each message
- ▶ Problem: **error propagation**
Bit error: 1-bit error in ciphertext leads to error in block and following block -> recovery after two blocks
Synchronization error (some bits are lost): no recovery anymore



uses key and previous block to encrypt,
encryption cannot be done in parallel since
it depends on the previous encryption

Cipherblock i

$$C_i = E_K(P_i \oplus C_{i-1})$$

$$P_i = C_{i-1} \oplus D_K(C_i)$$

Plaintext block i

Ciphertext block i-1

XOR

▶ ECB vs. CBC

- ▶ Popular error with encryption: Use wrong mode
- ▶ **ECB is bad**, CBC is better
- ▶ But much better is GCM (comes later)



ECB



CBC



Attention when choosing the initialization vector

- ▶ Initialization vector
 - ▶ Does not have to be kept secret (just send it along with the ciphertext)
 - ▶ Should be random (e.g. SecureRandom)
 - ▶ Should be different for each message
 - ▶ Is the same size as the block size, for AES **always** 128 bit
- ▶ Popular errors
 - ▶ Use Key as IV
 - ▶ This may cause the key to be read in the worst case [1]
 - ▶ Set IV to 0 or other fixed value
 - ▶ IV brings randomness into the system - therefore roll dice randomly!
 - ▶ Generate IV from password
 - ▶ Password has to be changed for each message to keep randomness

[1] <https://crypto.stackexchange.com/questions/16161/problems-with-using-aes-key-as-iv-in-cbc-mode>

▶ Operating mode CTR Counter Block Mode

- ▶ For each block a new unpredictable keystream block is calculated:
Keystream block = IV(Nonce) + current counter + encryption key

- ▶ Advantages:

Nonce = Number used only once

- ▶ Key does not depend on the key of the previous block
- ▶ Encryption and decryption can be performed in parallel
- ▶ Key stream can be precomputed (when operating as a stream cipher)
- ▶ Bit errors affect only one block

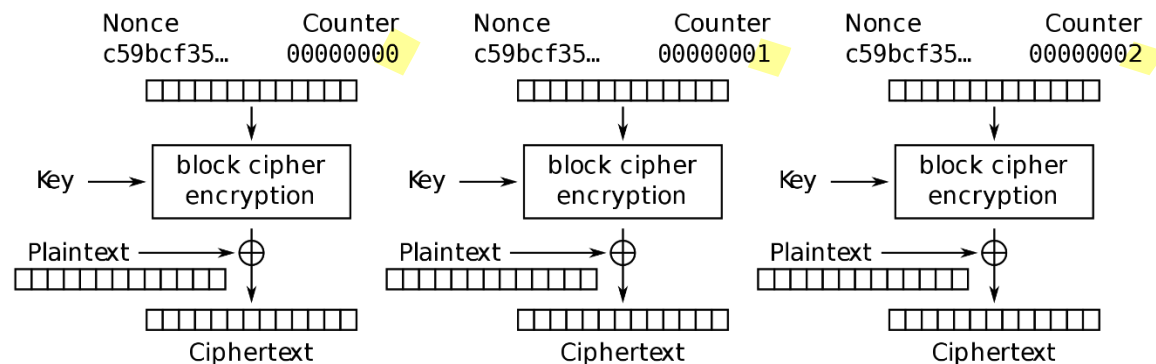
$$C_i = P_i \oplus E_K(ctr_i)$$

Cipherblock i

Plaintext i

XOR

Encryption with Key k and Counter i



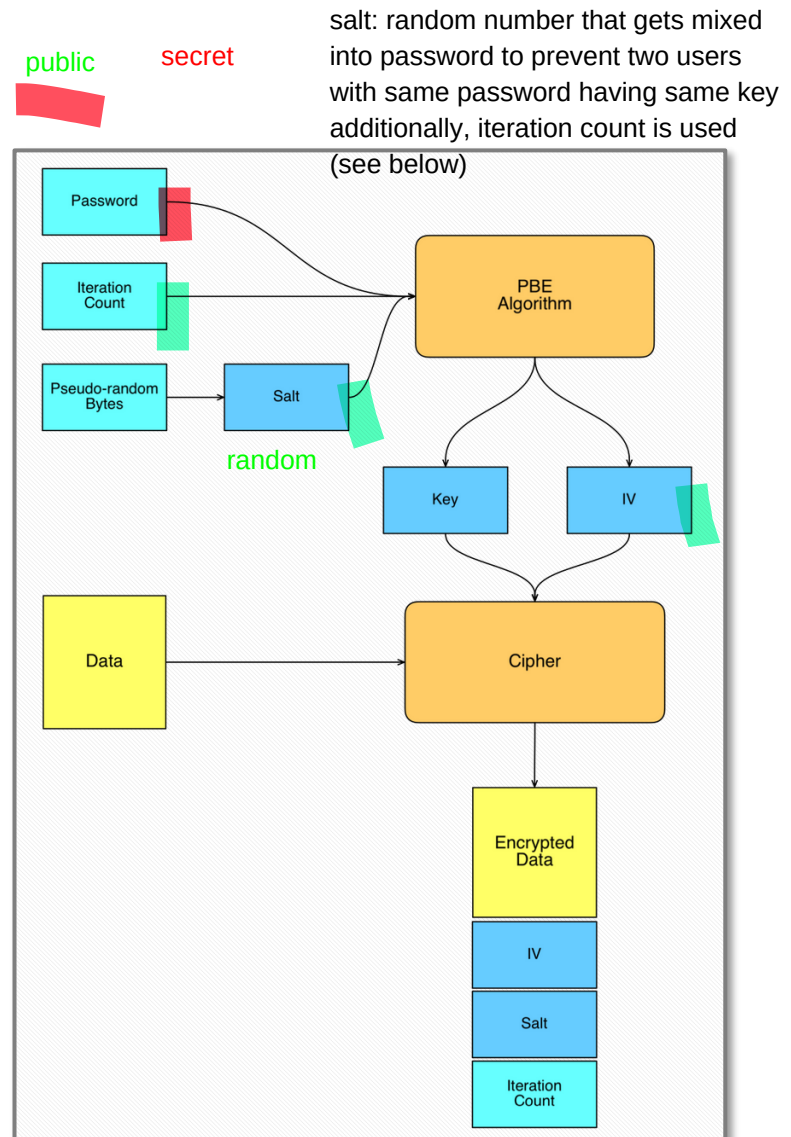
Counter (CTR) mode encryption



Password Based Encryption (PBE)

- ▶ PBE allows to generate cryptographic keys from passwords
 - Public Key Cryptographic Standard
- ▶ There are standardized methods
 - ▶ **PKCS#5 using PBKDF2** password based key derivation function (Password-Based Key Derivation Function2) a standardized function to derive a key from a password
 - ▶ **PKCS#12** defines a file format that is used to store private keys with the associated certificate in a password-protected manner.
- ▶ **Important:** The password must be good (high entropy) otherwise attacks are possible (e.g. dictionary attack).
- ▶ With the derived key you can apply a suitable symmetric cipher

problem: if two users have the same password, then they have the same encryption but solved via salt and iteration count (see above)



Schematic representation of PBE

Source: <http://www.crypto-it.net/eng/theory/pbe.html>

▶ Application of cryptography is difficult

- ▶ I use AES/CBC, all good? **NO!**
 - ▶ Attacker can modify ciphertext and thus influence plaintext
 - ▶ Attacker can perform **Padding Oracle Attack** and read plaintext, see <https://www.arxumpathsecurity.de/blog/2019/10/16/cbc-mode-is-malleable-dont-trust-it-for-authentication>
- ▶ Other popular error
 - ▶ Never use CBC, CTR, etc. without authentication!
 - ▶ Otherwise attacker can read / modify plaintext
- ▶ That's why: Secure encryption with authentication
 - ▶ Authenticated encryption provides integrity of the encrypted data secure check sum
 - ▶ Use a MAC (Message Authentication Code) for authentication
 - ▶ MAC = Hash function using a secret key

MAC is discussed in more detail
in the Digital Signatures chapter



Chapter 2.2: Authenticated Encryption

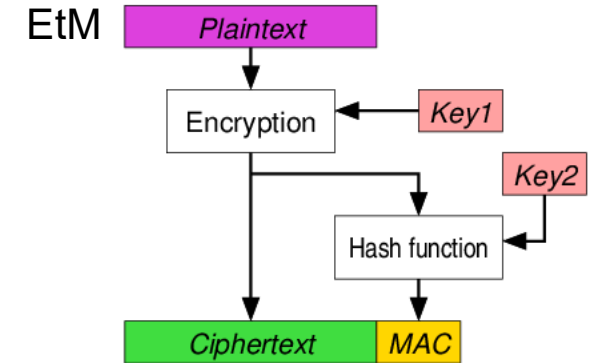
- Encrypt + MAC
- GCM
- AEAD
- ChaCha20, Poly1305

▶ Authenticated Encryption

- Encryption methods are called **authenticated** if not only the **confidentiality** but also the **integrity** of the data to be encrypted is protected.

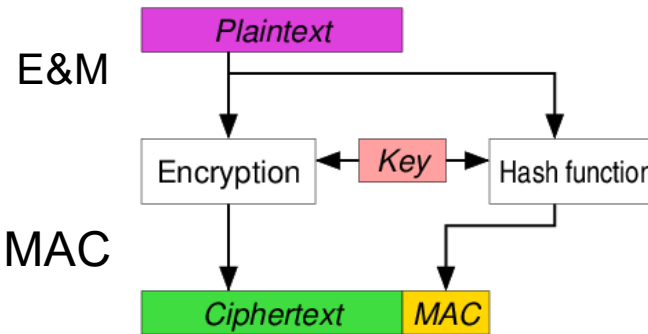
- Encrypt-then-MAC**

- MAC on ciphertext
- High Security** if both keys are different



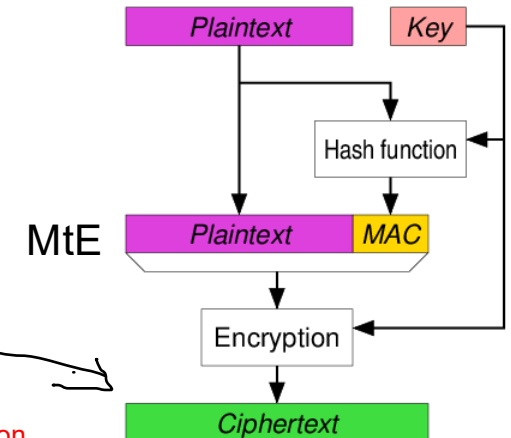
- Encrypt-and-MAC**

- MAC on plaintext
- Plaintext is encrypted without MAC



- MAC-then-Encrypt**

- MAC on plaintext
- MAC and plaintext are encrypted



Quelle: https://en.wikipedia.org/wiki/Authenticated_encryption



Authenticated Encryption

▶ Encrypt-then-MAC

- ▶ AES+CBC+HMAC-SHA256 CBC is block cipher

<https://www.mkammerer.de/blog/encrypt-something-with-aes-how-hard-can-it-be/>

- ▶ ChaCha20 (encrypt) + Poly1305 (MAC) ChaCha20 is stream cipher

<https://github.com/phxql/chacha20-poly1305-java>

▶ Popular errors

- ▶ Use same key for AES and HMAC
 - ▶ Is ok in certain cases, if possible: don't do it

- ▶ MAC-then-encrypt

- ▶ AES(HMAC(Plaintext) || Plaintext)
 - ▶ There are padding attacks (e.g. with SSL)

- ▶ MAC-and-encrypt

- ▶ AES(plaintext) || HMAC(plaintext)
 - ▶ No integrity on ciphertext

▶ Operating mode **GCM** Galois Counter Mode

▶ Authenticated encryption mode with associated data (AEAD)

- ▶ Fast, as parallelization is possible
- ▶ Can also be used as stand-alone MAC
- ▶ Accepts IV (nonce) of any length
- ▶ Auth Tag contains **Auth Data**, IV and ciphertext

e.g. TLS header information from the sender to the receiver

only key is private

E_K Encryption with Key k

$\text{Len}(A) = \text{len}(\text{Auth Data})$

$\text{Len}(C) = \text{len}(\text{Ciphertext})$

Mult H Galois field multiplications

authenticated encryption

▶ Video illustrating GCM

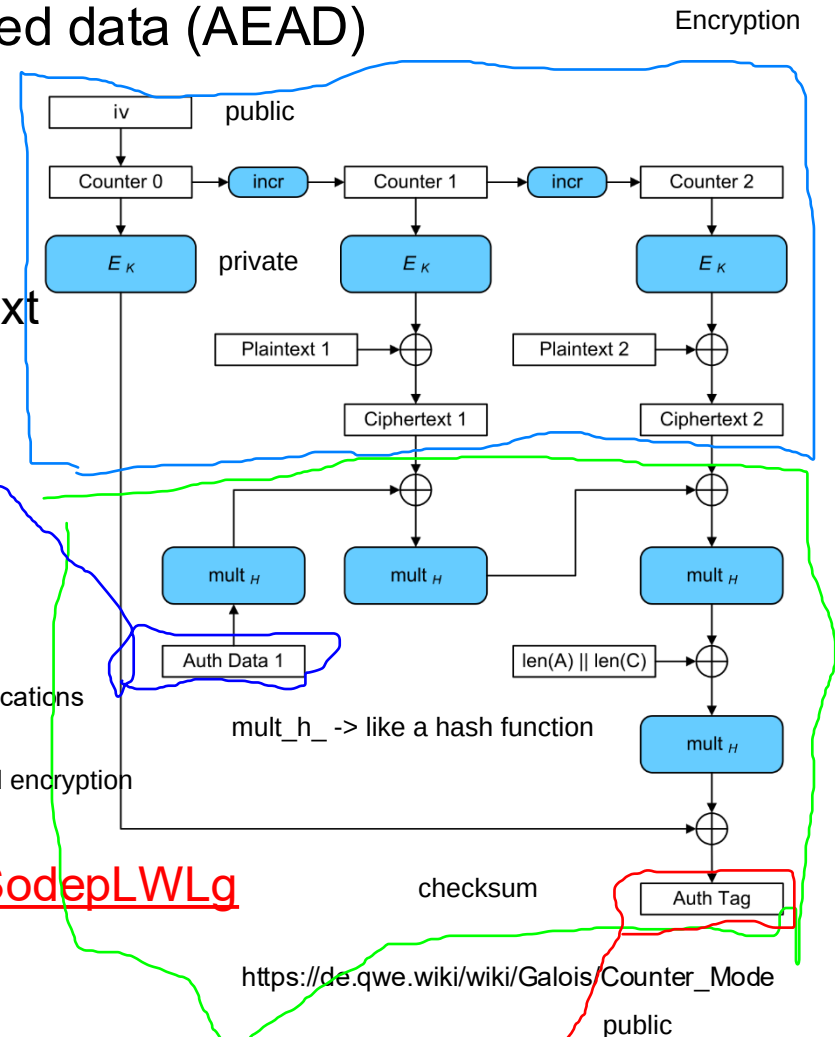


<https://www.youtube.com/watch?v=R2SodepLWLg>

https://de.qwe.wiki/wiki/Galois_Counter_Mode

public

checksum





AEAD Authenticated Encryption with Associated Data

security goals

- ▶ AEAD combines confidentiality, authenticity and integrity
 - ▶ is automatically secure against ciphertext modification
 - ▶ Combine encryption and MAC into one integrated protocol
 - ▶ e.g. AES-GCM
 - ▶ Uses a 96 bit nonce
 - ▶ Nonce (IV) need not be random, but must NEVER be reused with same key
 - ▶ Further variant AES-EAX

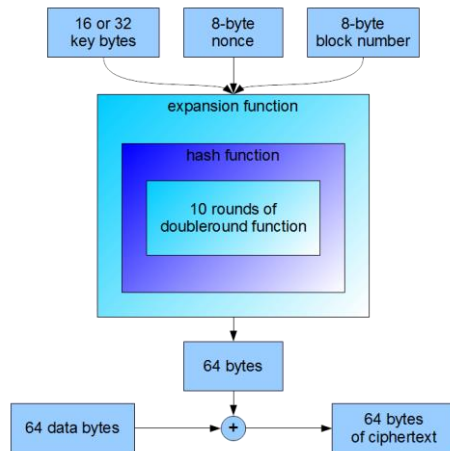
- ▶ Video illustrating AEAD
 - ▶  <https://www.youtube.com/watch?v=od44W45sCQ4>



AEAD with stream cipher ChaCha20 and checksum Poly1305

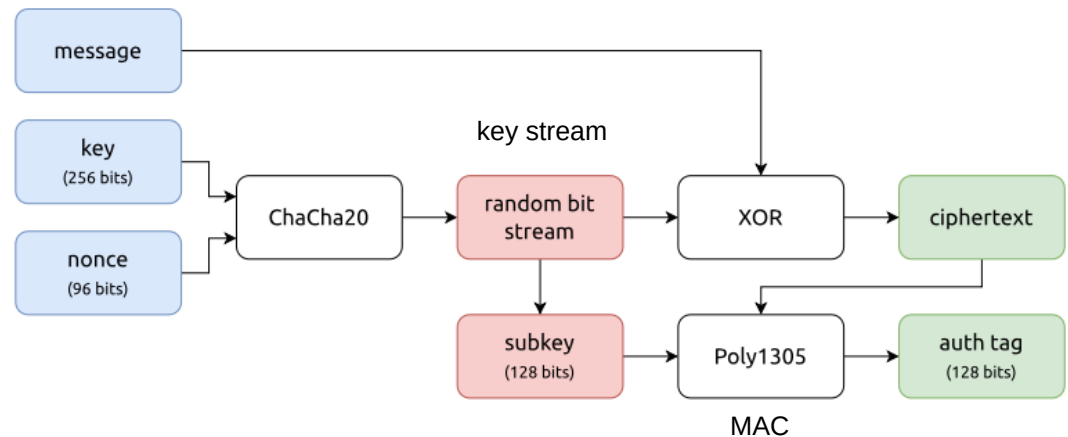
- ▶ ChaCha20
 - ▶ Symmetric stream cipher based on Salsa20 by Daniel Bernstein
 - ▶ Implementation easier than AES-GCM
 - ▶ Used in IPSec, TLS, OpenSSL, OpenSSH

- ▶ Poly1305 is a very fast MAC



Block Diagram of Salsa20 Algorithm

<http://www.crypt-it.net/eng/symmetric/salsa20.html>



ChaCha20 Poly1305 Encryption scheme

<https://andrea.corbellini.name/2023/03/09/authenticated-encryption/>



Chapter 2.3: Base64 Encoding



Base64 Encoding

Codierung -> Abbildungsvorschrift

Base64 is an encoding and not an encryption!

encoding: transform a character or bit info to another, reversible and not hidden

- ▶ For transmission of binary data in sequences of 8-bit bytes across channels that only reliably support text content, e.g. the World Wide Web.
- ▶ Representation of binary data with 64 printable ASCII characters.
- ▶ Principle:
 - ▶ Split 24 bits into 4 parts of 6 bits
 - ▶ Extend each 6 bit sequence to 8 bits
- ▶ Disadvantage: message becomes 33% longer
- ▶ Application:
 - ▶ Keys, signatures and certificates are often stored or transmitted BASE64 encoded.
 - ▶ E-Mail Attachments



Example for Base64 Encoding

Base64 Alphabet

Wert	Zeichen	Wert	Zeichen	Wert	Zeichen	Wert	Zeichen
0	A	17	R	34	i	51	z
1	B	18	S	35	j	52	0
2	C	19	T	36	k	53	1
3	D	20	U	37	l	54	2
4	E	21	V	38	m	55	3
5	F	22	W	39	n	56	4
6	G	23	X	40	o	57	5
7	H	24	Y	41	p	58	6
8	I	25	Z	42	q	59	7
9	J	26	a	43	r	60	8
10	K	27	b	44	s	61	9
11	L	28	c	45	t	62	+
12	M	29	d	46	u	63	/
13	N	30	e	47	v	64	=
14	O	31	f	48	w		
15	P	32	g	49	x		
16	Q	33	h	50	y		

Beispiel abcde			
Quelle	Binärdarstellung	Base64	
a	01100001		
b	01100010		
c	01100011		
24-Bit	011000010110001001100011		
6-Bit	011000	24	Y
	010110	22	W
	001001	9	J
	100011	35	j
d	01100100		
e	01100101		

24-Bit	011001000110010100_____		
6-Bit	011001	25	Z
	000110	6	G
	010100	20	U
	_____		=
YWJjZGU=			



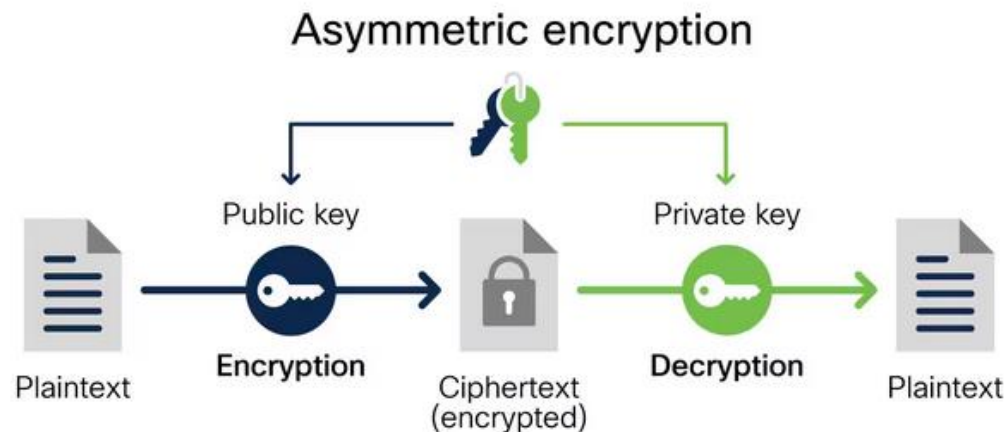
Chapter 2.4: Asymmetric Encryption

- RSA
- Elliptic Curve
- Hybrid Encryption
- ECIES



Asymmetric encryption

- ▶ Based on the use of **key pairs** (private and public key)
- ▶ There are fewer methods because they are more difficult to construct (they are based on problems of algorithmic number theory)
- ▶ Disadvantage: **High computing effort**
- ▶ **Advantage: Key exchange no problem**
- ▶ Examples: RSA, ECIES (Elliptic Curve), DLIES (Discrete Logarithm)

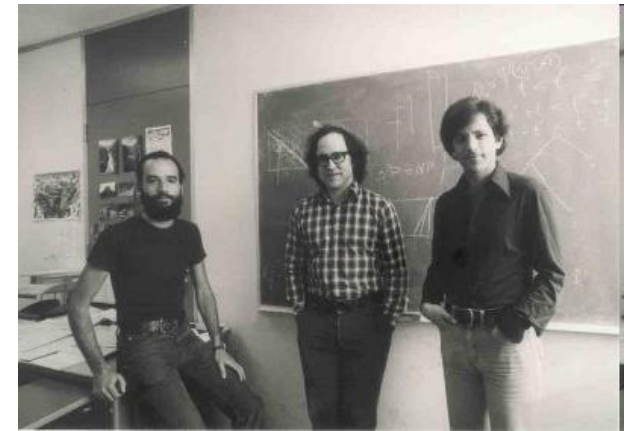


Source: <https://www.cisco.com/c/en/us/products/security/encryption-explained.html#~q-a>



RSA (Rivest-Shamir-Adleman)

- ▶ is an asymmetric algorithm
- ▶ is easy to understand and implement, popular, license-free
- ▶ To find the secret key from the public one, the prime factors of a number n must be found.
- ▶ Attack: Factoring large numbers is very difficult.
- ▶ key lengths of 3000 are currently required
- ▶ The implementations are much slower than AES.





RSA Algorithmus

1. choose 2 large ($> 2^{512}$) prime numbers p and q
2. calculate RSA module $n=p*q$
3. select $e < n$ with $\text{GCD}(e, (p-1)(q-1))=1$
4. select d so that $ed \bmod (p-1)(q-1) = 1$
5. public key is (e, n) , private key is (d, n)

uses public key

Encrypt: $c = m^e \bmod n$

Decrypt: $m = c^d \bmod n$
uses private key

private; only one person can use it



Elliptic Curve Cryptography

better than RSA

▶ Example for a curve equation:
 $y^2 = x^3 - 4x^2 + 10$

▶ Advantages of ECC:

- ▶ shorter key lengths as RSA
- ▶ this makes it faster
- ▶ safe at quantum computers

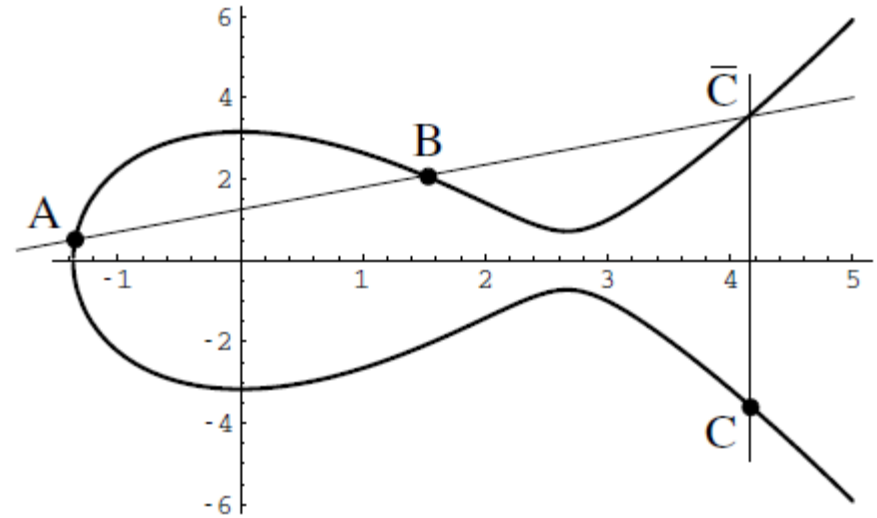
▶ Procedure: several passes with straight lines through two points and reflection of the third intersection

▶ Public Key: start and end point, Private Key: number of rounds

▶ Video for illustration



<https://www.youtube.com/watch?v=dCvB-mhkT0w>

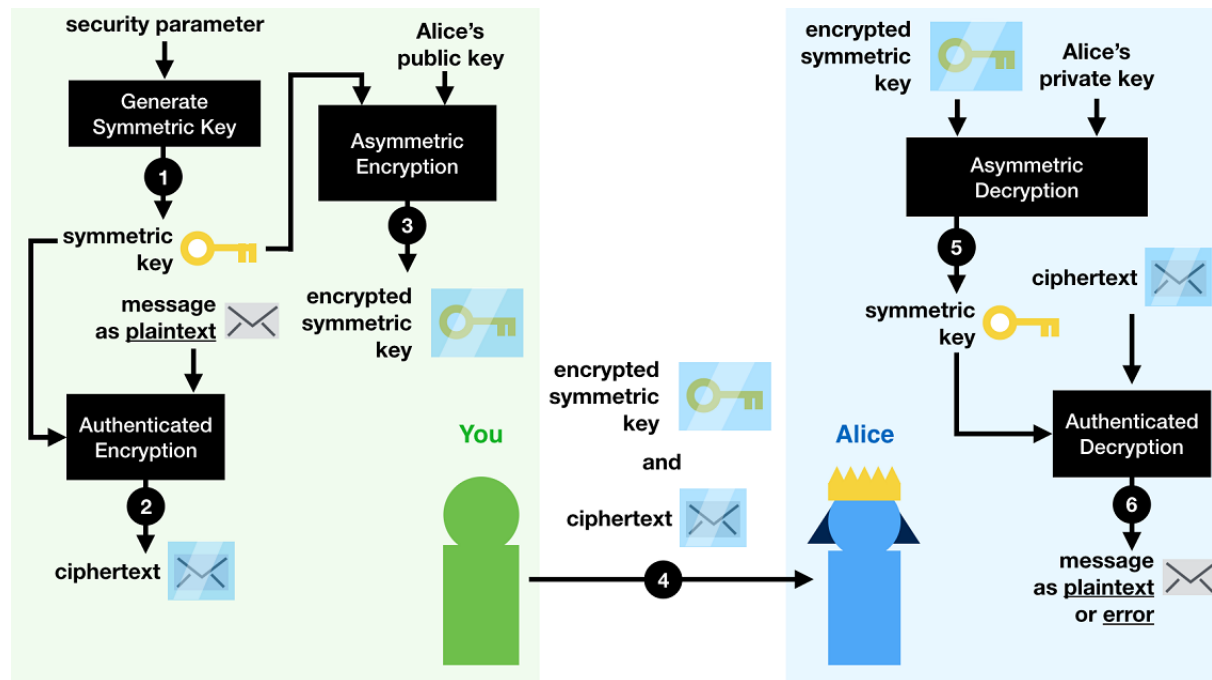


Source:: Angewandte Kryptographie, W.Ertl, Hanser



Hybrid Encryption

- ▶ Coupling symmetric and asymmetric encryption
- ▶ The message is first symmetrically encrypted (session key) and then the session key is encrypted asymmetrically
- ▶ Advantage: High speed of encryption and secure key exchange

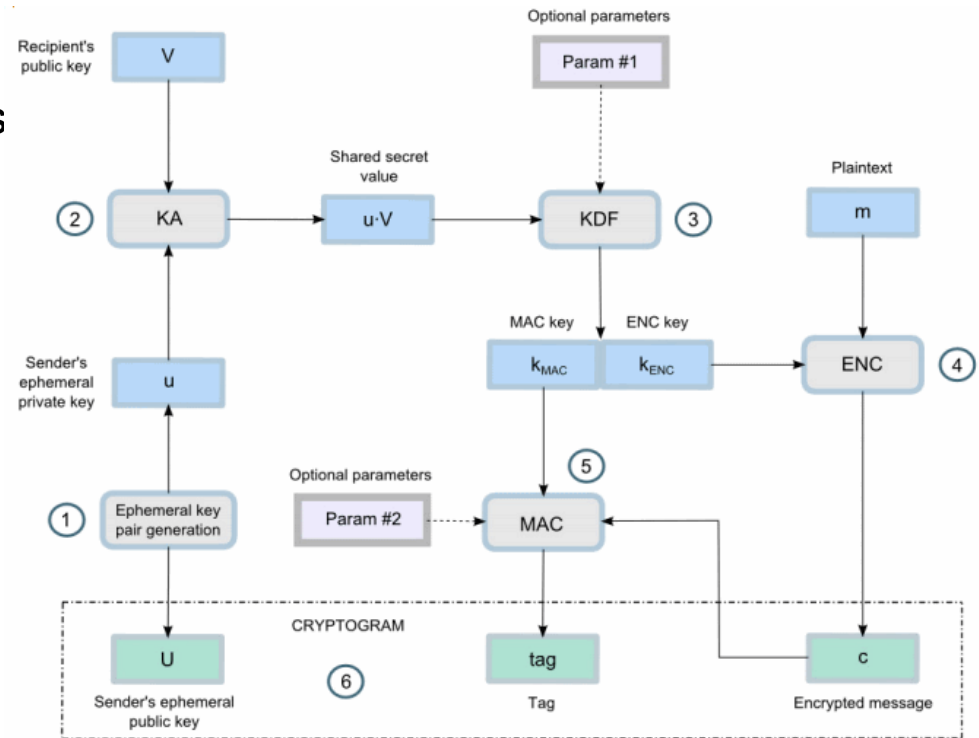


Source: Real-World Cryptography, David Wong, Manning Publications (2021)



ECIES Elliptic Curve Integrated Encryption Scheme

- ▶ **Hybrid encryption method** that combines ECC asymmetric cryptography with symmetric cipher and MAC
- ▶ ECIES is not a concrete algorithm but a framework
- ▶ You can use different algorithms (AES-CTR, AES-GCM, ChaCha20-Poly1305, ...)



KA Key Agreement (e.g. ECDH)
 KDF Key Derivation Function (e.g. cryptographic hash)
 ENC Encryption

- ▶ Further details can be found at

<https://medium.com/asecuritysite-when-bob-met-alice/go-public-and-symmetric-key-the-best-of-both-worlds-ecies-180f71eebf59>

https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Publikationen/TechnischeRichtlinien/TR02102/BSI-TR-02102.pdf?__blob=publicationFile&v=12





Chapter 2.5: Practical aspects of encryption

- Open Design
- Compression
- Export restrictions
- Storage formats
- Attacks



Practical aspects of encryption



- ▶ How do I build cryptography into my applications?
- ▶ In which format do I store encrypted data?
- ▶ What attacks are there on encryption?
- ▶ What key lengths and procedures are currently required?
- ▶ How do I create suitable keys?
- ▶ How do I get cryptographically secure random numbers?
- ▶ What do I have to consider when managing keys?
- ▶ How do I exchange keys securely?



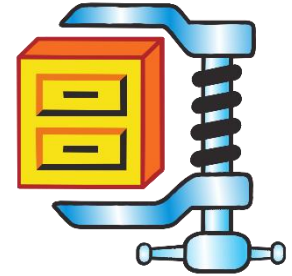
How do I build cryptography into my applications?

- ▶ Never do **"Security by obscurity"**:
 - ▶ Security by obscurity: System is a kind of black box, parts of the process or algorithm must be kept secret.
- ▶ Better apply **Open Design**:
 - ▶ The security of a mechanism should not depend on the secrecy of its design or implementation.
- ▶ **Kerckhoffs principle**:
 - ▶ The security of a cryptographic method may depend solely on the key used
 - ▶ everything else (including the algorithm itself) should be considered public knowledge.
- ▶ Do not use your own cryptography !!!
 - ▶ But use proven crypto libraries



What else do I have to consider?

▶ Compression – Encryption – Error Detection



- ▶ the right order is important
- ▶ **compression** makes sense because
 - ▶ encryption becomes **faster** when compression is fast
 - ▶ cryptanalysis (based on the exploitation of **redundancy in plain text**) is made more difficult
- ▶ an additional signature or checksum on encrypted data makes sense by **recognizing manipulations of the cipher text**
- ▶ When using cryptography in software products, **export restrictions** and **patents** must be respected
 - ▶ cryptography falls under the Weapons Act
 - ▶ Bureau of Industry and Security (<https://www.bis.doc.gov/>) regulates in USA export and import of technologies that use cryptography

review

first: compression (reduces data that needs to be encrypted > faster), only works on regular data (not on encrypted)

second: encryption

last: checksum (on cipher text) > verifies





A format for storing encryption: XML Encryption Standard

readable -> you see what you store

- ▶ W3C Recommendation (2013) <https://www.w3.org/TR/xmlenc-core/>
 - ▶ Describes the XML syntax for storing encrypted documents in XML.
 - ▶ Used in Web Services and SAML Identity Management
- ▶ There are the following encryption options
 - ▶ encryption of the entire XML document
 - ▶ encryption of a single element and its sub-elements
 - ▶ encryption of the contents of an XML element
 - ▶ multi-recipient encryption
- ▶ There are the specific XML elements for encryption
 - ▶ *<EncryptedData>*, *<EncryptionMethod>*, *<CipherData>*, *<CipherValue>*
- ▶ Advantage of XML encryption:
 - ▶ human readable format,
 - ▶ well-suited for understanding what needs to be saved in order to decrypt the data



Sample for XML Encryption: XML document with payment info

```
<PaymentInfo xmlns='http://example.org/paymentv2 '>
  <Name>John Smith</Name>
  <CreditCard Limit='5,000' Currency='USD '>
    <Number>4019 2445 0277 5567</Number>
    <Issuer>Example Bank</Issuer>
    <Expiration>04/02</Expiration>
  </CreditCard>
</PaymentInfo>
```

Task: Encrypt the **credit card information** for secure transport



XML document hybrid encrypted with public key of recipient

```
<PaymentInfo xmlns='http://example.org/paymentv2'>
  <Name>John Smith</Name>
  <EncryptedData Type='http://www.w3.org/2001/04/xmlenc#Element'
    xmlns='http://www.w3.org/2001/04/xmlenc#'>
    <EncryptionMethod Algorithm=
      'http://www.w3.org/2001/04/xmlenc#tripledes-cbc' />
    <KeyInfo xmlns='http://www.w3.org/2000/09/xmldsig#'>
      <EncryptedKey xmlns='http://www.w3.org/2001/04/xmlenc#'>
        <EncryptionMethod Algorithm='http://www.w3.org/2001/04/xmlenc#rsa-1_5' />
        <KeyInfo xmlns='http://www.w3.org/2000/09/xmldsig#'>
          <KeyName>Sally Doe</KeyName>
        </KeyInfo>
        <CipherData>
          <CipherValue>yMTEyOTA1M...</CipherValue>
        </CipherData>
      </EncryptedKey>
    </KeyInfo>
    <CipherData>
      <CipherValue>ydUNqHkMrD...</CipherValue>
    </CipherData>
  </EncryptedData>
</PaymentInfo>
```

unencrypted content

processing information

3DES/CBC

public key

key management hooks

encrypted key for secure transport

encrypted data

payment information



Another format for storing encryption: PKCS#7 Enveloped Data

- ▶ used in S/MIME
- ▶ can be combined with signatures: *Signed and Enveloped Data*
encrypted
- ▶ syntax of *Enveloped Data*

We will learn more details in
the chapter Digital Signatures

```
EnvelopedData ::= SEQUENCE {  
    version                Version,  
    recipientInfos          RecipientInfos,  
    encryptedContentInfo    EncryptedContentInfo  
}  
RecipientInfos ::= SET OF RecipientInfo  
EncryptedContentInfo ::= SEQUENCE {  
    contentType            ContentType,  
    contentEncryptionAlgorithm    ContentEncryptionAlgorithmIdentifier,  
    encryptedContent [0] IMPLICIT EncryptedContent OPTIONAL  
}  
EncryptedContent ::= OCTET STRING
```



Attacks on encryption, cryptanalysis



- ▶ Cryptanalysis: Mathematically analytical
 - ▶ Known Ciphertext, Known Plaintext, Chosen Plaintext, Chosen Ciphertext
only and CT decrypt
- ▶ Brute Force
- ▶ Social Engineering
- ▶ Search for vulnerabilities in the overall system in order to obtain keys
- ▶ Side Channel Attack
 - ▶ attack on a cryptographic system that exploits the results of physical measurements on the system (for example, energy consumption, electromagnetic radiation, time consumption of an operation) to gain insight into sensitive data.
- ▶ Fault Attack
 - ▶ attack on a cryptographic system in which the attacker uses or actively causes a faulty execution of a cryptographic operation.
- ▶ Man-in-the-middle attack



What key lengths and procedures are currently required?

- ▶ Key Length – Period of Validity
 - ▶ symmetric methods:
 - ▶ attacks via Brute Force
 - ▶ length at least 256
 - ▶ asymmetric methods:
 - ▶ attacks via mathematical methods
 - ▶ length at least 3000 (RSA), 250 (ECC)
 - ▶ Cryptosystems must be built in such a way that it is possible to change the methods and key lengths.
- ▶ This information must be reviewed regularly
- ▶ An important source is the BSI



Source for currently recommended procedures and key lengths:

https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Publikationen/TechnischeRichtlinien/TR02102/BSI-TR-02102.pdf?__blob=publicationFile&v=15



Chapter 2.6: Key Management

- Storage
- Key Exchange
- Diffie Hellman



Key Management



- ▶ Cryptographic methods are only as secure as the keys are protected
- ▶ A cleanly implemented key management is required for this
- ▶ Important **Key Management** features
 - ▶ generation of keys
 - ▶ storage of keys
 - ▶ withdrawal of invalid or compromised keys
 - ▶ secure transmission – exchange of keys
 - ▶ backup copies of keys
 - ▶ destruction of keys

key store was protected by password based encryption (first pswd)
second pswd was used for integrity
(cf. exercise 01.04.)



Key Management: Storage



▶ Where do I store keys, credentials, tokens?

- ▶ in source code? -> never!!
- ▶ in configuration files, environment variables:
either strict access control or encrypt
- ▶ Password Based Encryption (**PBE**)
 - ▶ Example: Java Keystore
- ▶ external keystores
 - ▶ **Vaults/Secret Manager** as a service or in a HSM (Hardware Security Module),
access via API with authentication and authorization and with audit logs
Example: Microsoft Azure Key Vault, AWS Secrets Manager
 - ▶ **Sealed Secrets** in git based on asymmetric crypto
<https://learnk8s.io/kubernetes-secrets-in-git>

very safe, also against physical manipulation, needs pin

- ▶ **Chip card** and **PIN**
- ▶ **Key splitting** (secret splitting, secret sharing: $n:m$ principle)

$n > m$

Split keys in n instances
(persons), m instances are
required to reconstruct keys

e.g. 5 people with 5 secrets,
only need 3 to reconstruct

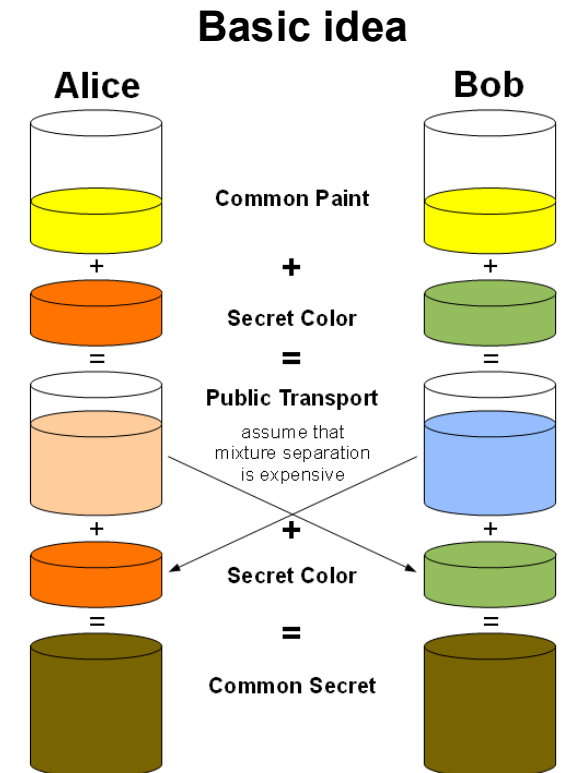


Key management: generate, withdraw/lock, transfer

- ▶ Key generation
 - ▶ good random numbers are required
 - ▶ they should be generated in a secure place (e.g. smart card, HSM)
- ▶ If a key is compromised, it must be exchanged
 - ▶ mark key as locked
 - ▶ replace key
 - ▶ re-encrypt all affected data
 - ▶ the actions **lock-exchange-encrypt** are also necessary if a key is no longer secure (encryption procedure insecure, length of key too short)
- ▶ Secure transmission - exchange of keys
 - ▶ mostly based on asymmetric cryptography (DH, ECDH, RSA and PKI)

▶ Diffie-Hellman key exchange method

- ▶ DH is a public key algorithm for the distribution of keys
- ▶ DH is not suitable for encryption
- ▶ Each communication partner calculates the common, secret session key for itself
- ▶ Thus, the key does not have to be transported over the network
- ▶ DH is based on the mathematical problem of the discrete logarithm
- ▶ The recommended key length is 3000 bits



https://commons.wikimedia.org/wiki/File:Diffie-Hellman_Key_Exchange.png



Diffie-Hellman key exchange



Protocol flow

- ▶ Alice and Bob agree on a large prime number n and a number g these are public
- ▶ Alice chooses a large random number x and sends $X = g^x \mod n$ private to Bob
- ▶ Bob chooses a large random number y and sends $Y = g^y \mod n$ private to Alice
- ▶ Alice calculates $k = Y^x \mod n$
- ▶ Bob calculates $k' = X^y \mod n$
- ▶ At the end $k' = k$ results in the same secret key

- ▶ **Public key:** n, g, X, Y
- ▶ **Private key:** x, y, k
- ▶ **Key length:** length of n in bit

i.e. combine public information with private

hybrid encryption: similar principle but sends over encrypted key but whole key is sent over network

and can be decrypted at later date if private key is obtained

diffie-hellman, on the other hand, cannot be reversed the same way



Elliptic Curve Diffie-Hellman ECDH

green: public
red: private

- ▶ the security is based on the difficulty of the Diffie-Hellman problem in elliptic curves
 - ▶ The key lengths is much shorter than for classical DH

p prime number
a,b curve parameter
P base point on curve
q order of P
i cofactor

▶ Protocol flow

- ▶ choose cryptographically strong EC system parameters (p, a, b, P, q, i), key length q should be at least 250 bits
- ▶ exchange system parameters and all following steps authentically
- ▶ A chooses equally distributed a random value $x \in \{1, \dots, q - 1\}$ and sends $Q_A = x \cdot P$ to B
- ▶ B chooses equally distributed random value $y \in \{1, \dots, q - 1\}$ and sends $Q_B = y \cdot P$ to A
- ▶ A calculates $x \cdot Q_B = xy \cdot P$
- ▶ B calculates $y \cdot Q_A = xy \cdot P$

▶ Video for illustration



<https://www.youtube.com/watch?v=yDXiDOJgxmg>



Key management: save, destroy



- ▶ Backup ^{or master key} **copies** of keys are usually necessary
 - ▶ for exceptional situations (**death**, termination of employment, vacation, ...) and loss
copy: where to store and who can access it?
 - ▶ Key **Recovery**: deposit for **key loss**
state wants access to encrypted data, so receives key > called key escrow
 - ▶ Key **Escrow**: deposit for **law enforcement**
 - ▶ **Secret Sharing** (e.g. Shamir Secret Sharing)
 - ▶ **Master Key**
 - ▶ **ATTENTION**: do not make backup copy of private signature keys

otherwise
signatures can be
faked

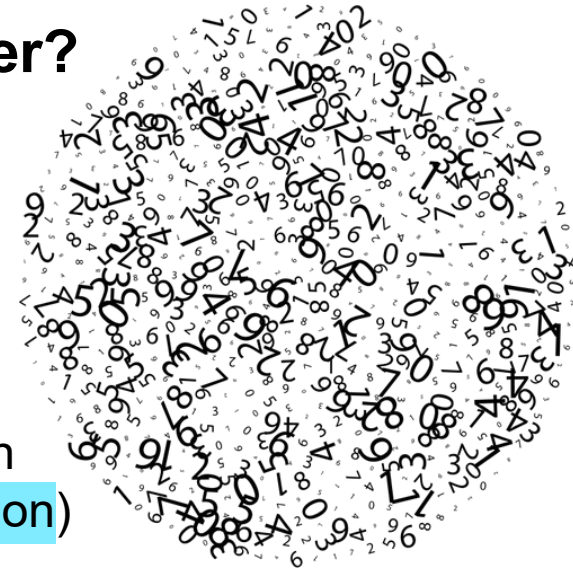
- ▶ Key **destruction**
 - ▶ **overwrite files** and **hard disk memory areas**, delete cache and swap areas
 - ▶ hardware may have to be destroyed



Chapter 2.7: Random Numbers



Randomness: How random is a random number?



- ▶ Initial situation
 - ▶ the quality of the encryption depends on the quality of the keys
 - ▶ the **entropy** is a measure of the disorder or randomness in a closed system (maximum entropy means equal distribution)
 - ▶ many other algorithms need random numbers (e.g. salt for hash)
IV
- ▶ Problem:
 - ▶ computers are deterministic machines, randomness is hard to get
- ▶ Solution:
 - ▶ collect as many sources of entropy as possible that are hard to predict (interrupts, user input, noise from sound card, etc.)

73735	45963	78134	63873
02965	58303	90708	20025
98859	23851	27965	62394
33666	62570	64775	78428
81666	26440	20422	05720

15838	47174	76866	14330
89793	34378	08730	56522
78155	22466	81978	57323
16381	66207	11698	99314
75002	80827	53867	37797

99982	27601	62686	44711
84543	87442	50033	14021
77757	54043	46176	42391
80871	32792	87989	72248
30500	28220	12444	71840



Cryptographic random numbers



- ▶ Properties of a good random number generator
 - ▶ it generates evenly distributed numbers
 - ▶ the numbers cannot be predicted
 - ▶ it has a long and complete cycle
- ▶ **Problem:** SW-implemented **PRNG's** (Pseudo Random Number Generator) are mostly fast but predictable. (e.g. mathematical random functions)
- ▶ Solutions:
 - ▶ **True RNG's** based on HW (like electronic noise) are expensive and complex.
 - ▶ PRNG mix different random sources (time, internal CPU counters, keystrokes, mouse movement etc.) as initial value and use a progressive function to calculate the random number
 - ▶ Cryptographic libraries offer PRNG:
 - .Net: System.Security.Cryptography.RandomNumberGenerator,
 - JCE: java.security.SecureRandom
 - ▶ use /dev/random (GNU/Linux) or getrandom(2) syscall

have enough entropy for normal user and most use cases



Example

```
private void random() {  
    SecureRandom random = new SecureRandom();  
  
    byte[] data = new byte[256 / 8];  
    random.nextBytes(data);  
    System.out.println(bytesToHex(data));  
  
    // Do not create a new SecureRandom, reuse it  
    byte[] moreData = new byte[256 / 8];  
    random.nextBytes(moreData);  
    System.out.println(bytesToHex(moreData));  
}
```

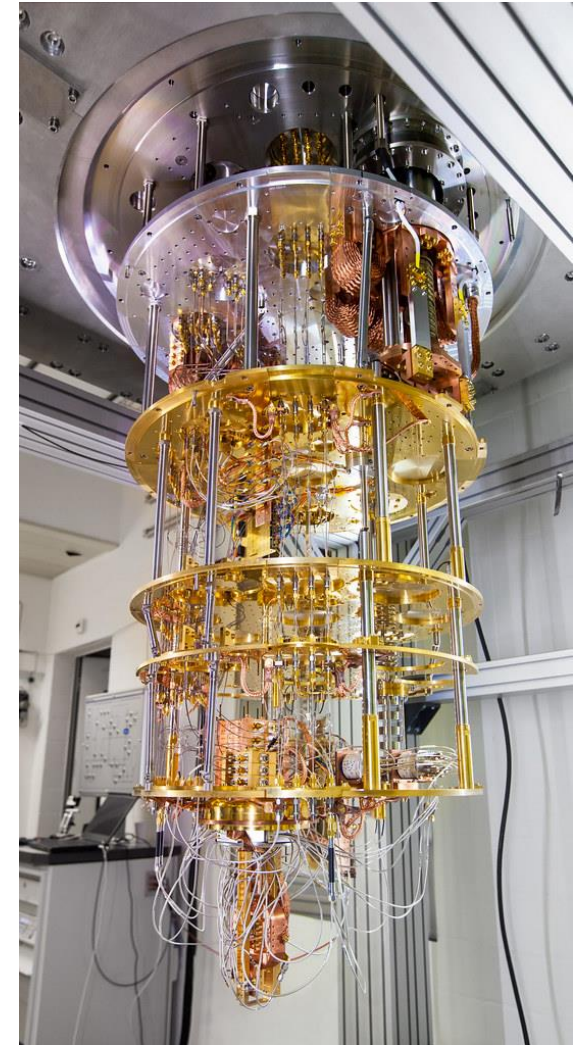


Chapter 2.8: Quantum Cryptography

▶ Quantum computing threatens cryptography

based on mathematical operations which could be reversed/inverted by quantum computing

- ▶ Quantum computers **endanger asymmetric procedures**
- ▶ **Symmetric methods are not affected so much**
(just increase key length to be safe) would still need to brute force each number so not much faster (cf. asymmetric)
- ▶ Quantum computers are **only theory so far**
 - ▶ but they will come in any case
 - ▶ secret services/organizations can collect data today and decrypt it in the future
 - ▶ the **replacement of today's cryptography and infrastructure is time consuming**
- ▶ There exist already algorithms to attack cryptography
 - ▶ Shor's algorithm
 - ▶ Grover's algorithm





Quantum secure cryptography

- ▶ Therefore, there are already approaches for quantum secure cryptography today
 - ▶ **PQC Post Quantum Cryptography**: mathematical problems that cannot be solved by QC (e.g. FrodoKEM, Classic McEliece)
 - ▶ **QKD Quantum Key Distribution**: secure key distribution (e.g. NewHope key exchange)
- ▶ Competition of NIST
 - ▶ NIST seeks quantum-resistant public-key algorithms
<https://csrc.nist.gov/projects/post-quantum-cryptography>
- ▶ Video about Quantum Cryptography
 - ▶ Mathematical explanation how quantum computers can crack current algorithms
 <https://www.youtube.com/watch?v=lvTqbM5Dq4Q&t=25s>



Conclusion

- ▶ Learning: Crypto is hard
 - ▶ applies not only to the algorithms, but also to their use
 - ▶ applies also to the chapter on hash, MAC, signatures
- ▶ Take the right library
 - ▶ Tink von Google <https://github.com/tink-crypto>
 - ▶ Bouncy Castle (<https://www.bouncycastle.org>)
 - ▶ Spring Security (<https://docs.spring.io/spring-security/reference/>)
 - ▶ LibSodium (<https://libsodium.gitbook.io/doc/>)
- ▶ Literature
 - ▶ <https://cryptobook.nakov.com/>
 - ▶ <https://www.garykessler.net/library/crypto.html>



Summary for encryption



- ▶ Symmetric encryption is fast.
- ▶ Hybrid encryption enables fast encryption with secure key exchange.
- ▶ The best encryption algorithm is of no use with poor key management.
- ▶ It is important to use the known methods correctly and also to pay attention to the details (random numbers, operating modes, initialization vector, checksums, ...).